Plataforma de monitoreo y observabilidad de infraestructura TI

INTEGRANTES: 1. Adam Fabricio Sánchez López

2. Rudi Choque Bautista

3. Brandon González Salazar

4.Luis Angel Caceres Claros

CARRERA: Ingeniería de Sistemas

MATERIA: Sistemas Operativos **DOCENTE**:

Diego Patrick Cárdenas Sejas

6 de octubre de 2026

Índice

1. Organización del equipo de trabajo 4
1.1 Estructura organizativa 4
1.2 Roles y responsabilidades 4
1.3 Distribución de tareas 5
1.4 Mecanismos de coordinación 5
1.5 Forma de trabajo colaborativo 5
2. Descripción del problema a resolver 5
2.1 Antecedentes 5
2.2 Contexto 6
2.3 Situación actual 6
2.4 Personas o sectores afectados 7
2.5 Relevancia del problema 7
3. Objetivos 8
3.1 Objetivo general 8
3.2 Objetivos específicos 8
4. Investigación y justificación tecnológica 9
4.1 Arquitectura tecnológica de la plataforma 9
4.2 Stack tecnológico implementado 10
4.3 Justificación técnica 11
Prometheus 11
Exporters 11
PostgreSQL 11
FastAPI 11
Docker Compose 12
Grafana 12
Alertmanager 12
5. Análisis de impacto y usuarios finales 12
5.1 Usuarios finales y beneficios 12
5.2 Precio propuesto y justificación 13
5.3 Factibilidad comercial en la ciudad 13
6. Estimación de costos y factibilidad del proyecto 13
6.1 Recursos tecnológicos 13

6.2 Recursos humanos 14
6.3 Otros recursos 14
6.4 Factibilidad del proyecto 15
7. Estado de implementación y avance del proyecto 15
7.1 Arquitectura implementada 15
7.2 Implementación de la infraestructura base 15
7.3 Implementación de la base de datos 16
7.4 Implementación del backend 16
7.5 Implementación del sistema de monitoreo 17
Node Exporter 17
Blackbox Exporter 17
SNMP Exporter 17
Instrumentación de FastAPI 17
7.6 Configuración de métricas y retención 17
7.7 Reglas de alerta implementadas 18
7.8 Estado actual y siguientes fases 18
8. Anexos 20
Anexo A. Diagrama de Gantt y planificación del proyecto 20
Representación simplificada del Gantt 21
Anexo B. 21
Anexo C. 22
Anexo D. 22
Anexo E. 23
Anexo F. 23
Anexo G. 23
Referencias 24

Plataforma de monitoreo y observabilidad de infraestructura TI

1. Organización del equipo de trabajo

1.1 Estructura organizativa

El proyecto será desarrollado mediante un equipo de trabajo organizado de acuerdo con las diferentes áreas que intervienen en el diseño e implementación de una plataforma de monitoreo de infraestructura TI.

Debido a que el proyecto involucra sistemas operativos, redes, bases de datos, desarrollo de software, monitoreo y administración de infraestructura, las actividades se distribuyen considerando las responsabilidades técnicas necesarias para completar cada fase.

La organización del equipo busca facilitar el trabajo colaborativo y mantener una separación clara entre las actividades de investigación, desarrollo, configuración, pruebas y documentación.

1.2 Roles y responsabilidades

Las responsabilidades principales del equipo comprenden:

- Investigación de tecnologías de monitoreo y observabilidad.
- Diseño de la arquitectura de la plataforma.
- Configuración de la infraestructura mediante Docker.
- Diseño e implementación de la base de datos.
- Desarrollo del backend y API.
- Configuración de Prometheus y los exporters.
- Configuración y validación de las reglas de alerta.
- Pruebas de funcionamiento.
- Documentación técnica y académica.

- Preparación de la presentación y demostración del proyecto.

La distribución exacta de las tareas puede variar durante el desarrollo de acuerdo con las necesidades de cada fase.

1.3 Distribución de tareas

El trabajo se desarrolla de manera progresiva. Las primeras actividades se concentran en investigación y diseño, mientras que las fases posteriores se orientan a la implementación, integración y validación.

La división por fases permite que cada integrante pueda participar en diferentes áreas y que las actividades puedan ser verificadas antes de avanzar a la siguiente etapa.

1.4 Mecanismos de coordinación

Para la coordinación del proyecto se utilizan herramientas de comunicación y colaboración digital. El código fuente y los cambios relacionados con el desarrollo se gestionan mediante un repositorio, mientras que la documentación y los materiales de apoyo se organizan digitalmente.

Las reuniones y revisiones periódicas permiten verificar el avance, identificar problemas y establecer las actividades siguientes.

1.5 Forma de trabajo colaborativo

El desarrollo se realiza de forma incremental (Schwaber y Sutherland, 2020). Cada fase tiene objetivos específicos y se procura validar los resultados antes de continuar.

También se utilizan herramientas de inteligencia artificial como apoyo para actividades de programación, documentación, análisis y resolución de problemas. Estas herramientas se consideran instrumentos de apoyo al desarrollo y no sustituyen la revisión y validación realizada por los integrantes del equipo.

2. Descripción del problema a resolver

2.1 Antecedentes

La infraestructura tecnológica de una organización está compuesta por diferentes elementos, como servidores, equipos de red, servicios y aplicaciones. Estos componentes deben mantenerse disponibles y funcionando correctamente para garantizar la continuidad de las operaciones.

Cuando la supervisión se realiza principalmente de manera manual, la identificación de problemas puede depender de que un administrador revise individualmente cada equipo o de que los usuarios informen cuando un servicio deja de funcionar.

Las herramientas de monitoreo permiten centralizar esta actividad mediante la recopilación periódica de información sobre el estado y rendimiento de los diferentes componentes de la infraestructura (Zabbix, s. f.).

2.2 Contexto

El proyecto se plantea dentro del área de administración de sistemas e infraestructura TI, considerando un escenario en el que una organización necesita supervisar diferentes recursos tecnológicos desde una plataforma centralizada.

La solución propuesta contempla servidores Linux, servidores Windows, dispositivos de red como routers y switches, además de servicios que puedan ser comprobados mediante diferentes mecanismos de monitoreo.

El proyecto tiene un enfoque académico y busca demostrar los principios fundamentales de monitoreo y observabilidad mediante una implementación funcional y controlada.

2.3 Situación actual

La ausencia de una plataforma centralizada puede dificultar la identificación temprana de problemas relacionados con el consumo de recursos, disponibilidad de dispositivos, estado de interfaces de red o funcionamiento de servicios.

Entre las principales dificultades se encuentran:

- Falta de información centralizada.
- Dificultad para consultar el comportamiento histórico.
- Detección tardía de fallas.
- Dependencia de verificaciones manuales.
- Falta de alertas automáticas.
- Dificultad para identificar tendencias de utilización de recursos.
- Necesidad de utilizar diferentes herramientas para supervisar distintos tipos de infraestructura.

Por este motivo, se plantea la implementación de una plataforma capaz de recolectar métricas, almacenarlas, visualizarlas y generar alertas cuando se presenten determinadas condiciones.

2.4 Personas o sectores afectados

Los principales usuarios de la solución serán:

- Administradores de sistemas.
- Administradores de redes.
- Personal encargado de infraestructura TI.
- Técnicos de soporte.
- Responsables de servicios tecnológicos.

Indirectamente, también se benefician los usuarios de los servicios tecnológicos, debido a que una detección más rápida de incidentes puede contribuir a reducir el tiempo durante el cual un servicio permanece afectado.

2.5 Relevancia del problema

El monitoreo constituye una actividad importante para la administración de infraestructura porque permite obtener información sobre el comportamiento de los sistemas antes y durante la aparición de incidentes (Kim et al., 2016).

Una plataforma de monitoreo no elimina las fallas, pero proporciona información que permite detectarlas, analizarlas y responder de manera más oportuna.

Por esta razón, el proyecto busca implementar una solución que permita demostrar el funcionamiento de un sistema de monitoreo centralizado y su aplicación en la administración de infraestructura TI.

3. Objetivos

3.1 Objetivo general

Diseñar e implementar una plataforma centralizada de monitoreo y observabilidad de infraestructura TI que permita recolectar, almacenar, visualizar y analizar métricas de servidores, dispositivos de red y servicios, facilitando la detección temprana de incidentes mediante reglas de alerta configurables.

3.2 Objetivos específicos

1. Diseñar una arquitectura de monitoreo que permita integrar servidores Linux, servidores Windows, dispositivos de red y servicios dentro de una plataforma centralizada.

2. Implementar una infraestructura de ejecución mediante Docker Compose que permita desplegar de manera reproducible los diferentes componentes de la plataforma.
3. Diseñar e implementar una base de datos PostgreSQL para administrar información estructural de usuarios, dispositivos, ubicaciones, servicios, reglas de alerta, incidentes y auditoría.
4. Desarrollar una API mediante FastAPI que permita administrar los recursos principales de la plataforma y proporcionar una interfaz de comunicación con los componentes de gestión.
5. Implementar Prometheus como sistema principal para la recolección, consulta y almacenamiento de métricas de series temporales.
6. Integrar exporters y mecanismos de monitoreo para obtener información de servidores Linux, servidores Windows, dispositivos de red mediante SNMP y disponibilidad de servicios.
7. Configurar reglas de alerta para detectar condiciones anormales relacionadas con utilización de CPU, memoria, almacenamiento, disponibilidad y tráfico de red.
8. Implementar mecanismos de visualización mediante dashboards que permitan analizar el estado actual y el comportamiento histórico de la infraestructura.
9. Evaluar el funcionamiento de la plataforma mediante pruebas controladas de disponibilidad, consumo de recursos y generación de alertas.
10. Documentar la arquitectura, configuración, pruebas y resultados obtenidos durante el desarrollo del proyecto.

4. Investigación y justificación tecnológica

4.1 Arquitectura tecnológica de la plataforma

La plataforma se diseñó siguiendo una arquitectura modular en la que cada componente cumple una función específica.

La arquitectura definida está compuesta principalmente por:

- Prometheus.
- Grafana.
- Alertmanager.
- FastAPI.
- PostgreSQL.
- Node Exporter.
- Windows Exporter.
- SNMP Exporter.
- Blackbox Exporter.
- Docker Compose.

El flujo general de monitoreo comienza en los dispositivos o servicios supervisados. Los exporters o mecanismos correspondientes exponen información que posteriormente es recolectada por Prometheus.

Prometheus mantiene las métricas de series temporales y permite realizar consultas y evaluar reglas de alerta. Grafana se utiliza para la visualización de información mediante dashboards, mientras que Alertmanager será utilizado para gestionar las alertas generadas por Prometheus.

FastAPI proporciona la API propia de la plataforma y PostgreSQL almacena la información estructural y administrativa del sistema.

La arquitectura puede representarse de manera general de la siguiente forma:

Dispositivos y servicios → Exporters/Monitoreo → Prometheus → Dashboards y Alertas

De forma paralela:

Usuario → Plataforma web → FastAPI → PostgreSQL

4.2 Stack tecnológico implementado

La Tabla 1 resume los componentes que conforman el stack tecnológico de la plataforma y la función principal de cada uno.

Tabla 1

Stack tecnológico de la plataforma

Componente	Tecnología	Función principal
Contenedores	Docker	Ejecución aislada de los componentes
Orquestación	Docker Compose	Administración del entorno
Backend	FastAPI	API y lógica de gestión
Base de datos	PostgreSQL	Información estructural y administrativa
Migraciones	Alembic	Control de cambios de la base de datos
Monitoreo	Prometheus	Recolección y almacenamiento de métricas
Servidores Linux	Node Exporter	Métricas del sistema Linux
Servidores Windows	Windows Exporter	Métricas del sistema Windows
Dispositivos de red	SNMP Exporter	Recolección de información mediante SNMP
Disponibilidad	Blackbox Exporter	Verificación de servicios y endpoints
Visualización	Grafana	Dashboards y análisis visual
Alertas	Alertmanager	Gestión y envío de notificaciones
Lenguaje backend	Python	Desarrollo de la API
Base de frontend	React/TypeScript	Interfaz propia de gestión

Nota. Elaboración propia.

La utilización de componentes especializados permite separar las responsabilidades del sistema. Prometheus se encarga de las métricas, PostgreSQL de la información estructurada y

FastAPI, desarrollado en Python (Python Software Foundation, s. f.), de la lógica de gestión de la plataforma.

4.3 Justificación técnica

Prometheus

Prometheus fue seleccionado como componente central de monitoreo porque está orientado específicamente a la recopilación y consulta de métricas de series temporales. Permite definir intervalos de recolección y utilizar PromQL para realizar consultas y construir reglas de alerta (Prometheus Authors, s. f.).

Exporters

Los exporters permiten adaptar la recolección de información a diferentes tipos de infraestructura.

Node Exporter permite obtener métricas relacionadas con sistemas Linux. Windows Exporter permite realizar una función equivalente en servidores Windows. SNMP Exporter permite obtener información de dispositivos compatibles con SNMP, como routers y switches.

Blackbox Exporter permite verificar la disponibilidad y respuesta de determinados servicios o endpoints.

PostgreSQL

PostgreSQL se utiliza para almacenar información estructurada de la plataforma (PostgreSQL Global Development Group, s. f.). No se utiliza como almacenamiento principal de las métricas de alta frecuencia.

Entre la información que administra se encuentran usuarios, dispositivos, ubicaciones, servicios, reglas, incidentes y registros de auditoría.

FastAPI

FastAPI permite construir una API REST para administrar los recursos propios de la plataforma (FastAPI, s. f.). Su función es complementar la infraestructura de monitoreo proporcionando una capa de gestión para usuarios y configuraciones.

Docker Compose

Docker Compose permite definir y ejecutar los diferentes componentes de la plataforma de manera reproducible (Docker, s. f.). Esto facilita el desarrollo, las pruebas y la configuración del entorno.

Grafana

Grafana se utilizará como herramienta de visualización para presentar métricas y evolución histórica mediante dashboards (Grafana Labs, s. f.).

Alertmanager

Alertmanager será utilizado para gestionar las alertas generadas por Prometheus, permitiendo posteriormente implementar mecanismos de agrupación, enrutamiento y notificación.

5. Análisis de impacto y usuarios finales

5.1 Usuarios finales y beneficios

La plataforma está orientada principalmente a personal técnico responsable de la infraestructura TI.

Los principales beneficios esperados son:

- Centralización de información.
- Supervisión continua.
- Consulta de métricas históricas.

- Detección temprana de problemas.
- Identificación de dispositivos no disponibles.
- Identificación de problemas de recursos.
- Visualización del comportamiento de la infraestructura.
- Generación de alertas.
- Apoyo a la toma de decisiones técnicas.

El sistema también puede servir como una herramienta de apoyo para identificar tendencias de consumo y determinar cuándo un recurso se aproxima a niveles que requieren atención.

5.2 Precio propuesto y justificación

Como propuesta académica, la plataforma está orientada a utilizar principalmente tecnologías de código abierto. Esto permite reducir los costos asociados a licencias de software.

Los costos principales estarían relacionados con la infraestructura utilizada para ejecutar el sistema, como servidores, almacenamiento, conectividad y mantenimiento.

En un escenario académico, la plataforma puede ejecutarse sobre máquinas virtuales o infraestructura disponible para el equipo, por lo que el costo directo de software puede mantenerse reducido.

5.3 Factibilidad comercial en la ciudad

La solución puede ser aplicable a pequeñas y medianas organizaciones que necesiten supervisar su infraestructura tecnológica sin depender necesariamente de plataformas comerciales de alto costo.

Sin embargo, el objetivo principal del presente proyecto no es desarrollar un producto comercial completo, sino demostrar la implementación y funcionamiento de una plataforma de monitoreo y observabilidad dentro del contexto académico.

6. Estimación de costos y factibilidad del proyecto

6.1 Recursos tecnológicos

Los principales recursos tecnológicos necesarios son:

- Computadores para desarrollo.
- Máquinas virtuales o servidores.
- Sistema operativo Linux.
- Docker y Docker Compose.
- PostgreSQL.
- Prometheus.
- Grafana.
- Exporters.
- Conectividad de red.
- Repositorio para control de versiones.

El proyecto puede desarrollarse utilizando infraestructura virtualizada, lo que permite reducir la necesidad de disponer de múltiples equipos físicos.

6.2 Recursos humanos

El proyecto requiere conocimientos relacionados con:

- Sistemas operativos.
- Redes.
- Programación.

- Bases de datos.
- Desarrollo de APIs.
- Administración de servidores.
- Monitoreo de infraestructura.
- Seguridad informática básica.

La utilización de herramientas de inteligencia artificial puede apoyar las actividades de desarrollo, investigación y documentación, aunque la validación final corresponde al equipo.

6.3 Otros recursos

También son necesarios recursos para:

- Documentación.
- Control de versiones.
- Pruebas.
- Reuniones del equipo.
- Preparación de la presentación.
- Elaboración de evidencias.

6.4 Factibilidad del proyecto

El proyecto es técnicamente factible debido a que los principales componentes utilizados son tecnologías disponibles y ampliamente utilizadas en ambientes de infraestructura y desarrollo.

Además, el avance obtenido hasta la Fase 4 demuestra que es posible ejecutar de forma conjunta la infraestructura base, la base de datos, el backend y los componentes iniciales de monitoreo.

La implementación realizada hasta este punto permite continuar con las siguientes fases sin modificar la arquitectura principal.

7. Estado de implementación y avance del proyecto

7.1 Arquitectura implementada

Durante el desarrollo del proyecto, la propuesta inicial fue concretándose hasta establecer una arquitectura definida.

La arquitectura implementada utiliza Docker Compose para ejecutar los componentes principales del sistema.

Actualmente se dispone de una infraestructura que integra:

- PostgreSQL.
- FastAPI.
- Prometheus.
- Node Exporter.
- Blackbox Exporter.
- SNMP Exporter.

Los componentes se comunican dentro del entorno de contenedores y permiten realizar las primeras pruebas de recolección y exposición de métricas.

7.2 Implementación de la infraestructura base

Se implementó Docker Compose como mecanismo para ejecutar los servicios principales.

Esta configuración permite iniciar el entorno de desarrollo de forma conjunta y facilita la reproducción de la infraestructura en diferentes entornos.

La infraestructura base constituye la primera capa sobre la que se implementaron posteriormente la base de datos, la API y el sistema de monitoreo.

7.3 Implementación de la base de datos

Se implementó PostgreSQL como sistema gestor de base de datos.

La estructura definida contempla entidades para:

- Usuarios.
- Ubicaciones.
- Dispositivos.
- Servicios.
- Reglas de alerta.
- Asignaciones de reglas.
- Incidentes.
- Notificaciones.
- Registros de auditoría.

La administración de los cambios estructurales se realiza mediante Alembic.

La base de datos tiene como objetivo almacenar información estructural y administrativa de la plataforma, mientras que las métricas de series temporales son administradas por Prometheus.

7.4 Implementación del backend

Se desarrolló un backend mediante FastAPI.

La API permite administrar los recursos principales de la plataforma y proporciona endpoints para operaciones relacionadas con dispositivos, servicios y otros elementos administrativos.

También se incorporaron mecanismos de validación, paginación y filtrado.

El backend dispone además de instrumentación para exponer métricas que pueden ser recolectadas por Prometheus.

De esta manera, la propia aplicación puede ser supervisada dentro de la infraestructura de monitoreo.

7.5 Implementación del sistema de monitoreo

Prometheus fue integrado como componente principal para la recolección de métricas.

Hasta la Fase 4 se integraron los siguientes mecanismos:

Node Exporter

Permite recolectar información relacionada con el sistema Linux utilizado durante las pruebas.

Entre las métricas disponibles se encuentran datos relacionados con CPU, memoria, almacenamiento y red.

Blackbox Exporter

Permite realizar comprobaciones de disponibilidad y respuesta de servicios mediante diferentes protocolos y mecanismos de prueba.

SNMP Exporter

Se configuró para permitir la obtención de métricas desde dispositivos de red compatibles con SNMP.

El monitoreo mediante SNMP permite trabajar con información relacionada con interfaces, tráfico y errores de dispositivos de red.

Instrumentación de FastAPI

La aplicación FastAPI expone métricas propias que pueden ser recolectadas por Prometheus.

Esto permite observar aspectos relacionados con la ejecución de la plataforma.

7.6 Configuración de métricas y retención

La configuración actual utiliza un intervalo de recolección de aproximadamente 15 segundos.

Este valor se encuentra dentro del requisito establecido de utilizar un intervalo no mayor a 30 segundos.

También se estableció una retención de aproximadamente 90 días para las métricas.

La utilización de Prometheus permite consultar tanto el estado actual como información histórica disponible durante el periodo de retención.

7.7 Reglas de alerta implementadas

Durante la Fase 4 se configuraron reglas destinadas a detectar diferentes condiciones de funcionamiento.

Entre los escenarios contemplados se encuentran:

- Alto consumo de CPU.
- Alto consumo de memoria.
- Alto uso de almacenamiento.
- Problemas de disponibilidad.
- Caída de servicios.
- Problemas de interfaces de red.
- Condiciones relacionadas con tráfico.

- Riesgo de agotamiento de almacenamiento.

En total, se configuraron 13 reglas de alerta durante esta fase.

Estas reglas forman la base del sistema de detección de incidentes. La integración completa del mecanismo de gestión y notificación mediante Alertmanager corresponde a una fase posterior.

7.8 Estado actual y siguientes fases

El estado actual del proyecto corresponde al cierre de la Fase 4. La Tabla 2 presenta el estado de cada una de las fases del proyecto.

Tabla 2

Estado de avance por fase del proyecto

Fase	Actividad	Estado
Fase 0	Investigación, definición del problema y arquitectura	Completada
Fase 1	Infraestructura base y Docker Compose	Completada
Fase 2	PostgreSQL y Alembic	Completada
Fase 3	FastAPI y API REST	Completada
Fase 4	Prometheus y exporters	Completada
Fase 5	Evaluación de agente propio	Pendiente/opcional
Fase 6	Alertmanager y notificaciones	Pendiente
Fase 7	Grafana y dashboards	Pendiente
Fase 8	Integración y seguridad	Pendiente
Fase 9	Pruebas finales	Pendiente
Fase 10	Documentación y presentación	Pendiente

Nota. Elaboración propia.

La Fase 5 relacionada con un agente propio se considera actualmente opcional, debido a que la arquitectura ya dispone de exporters especializados para diferentes tipos de infraestructura.

Las siguientes actividades prioritarias corresponden a la integración de Alertmanager, la implementación de dashboards con Grafana, la integración completa de los componentes y las pruebas finales.

8. Anexos

Anexo A. Diagrama de Gantt y planificación del proyecto

El siguiente cronograma, presentado en la Tabla 3, representa la planificación general del proyecto desde las actividades iniciales hasta la etapa de presentación.

Las fechas deben ajustarse a las fechas reales registradas por el equipo en caso de existir diferencias con esta planificación de referencia.

Tabla 3

Cronograma de actividades del proyecto

Actividad	Inicio	Fin	Estado
Investigación inicial y definición del problema	03/08/2026	14/08/2026	Completada
Diseño de arquitectura	10/08/2026	14/08/2026	Completada
Fase 1: Infraestructura y Docker Compose	15/08/2026	23/08/2026	Completada
Fase 2: PostgreSQL y Alembic	24/08/2026	26/08/2026	Completada
Fase 3: FastAPI y API REST	27/08/2026	30/08/2026	Completada
Fase 4: Prometheus y exporters	04/09/2026	06/09/2026	Completada
Fase 5: Evaluación de agente propio	Octubre 2026	Octubre 2026	Pendiente
Fase 6: Alertmanager y notificaciones	Octubre 2026	Octubre 2026	Pendiente
Fase 7: Grafana y dashboards	Octubre 2026	Noviembre 2026	Pendiente
Fase 8: Integración y seguridad	Noviembre 2026	Noviembre 2026	Pendiente
Fase 9: Pruebas finales	Noviembre 2026	Noviembre 2026	Pendiente
Fase 10: Documentación y presentación	Noviembre 2026	Diciembre 2026	Pendiente

Nota. Elaboración propia.

Representación simplificada del Gantt

La Tabla 4 muestra de forma simplificada los meses de ejecución de cada actividad.

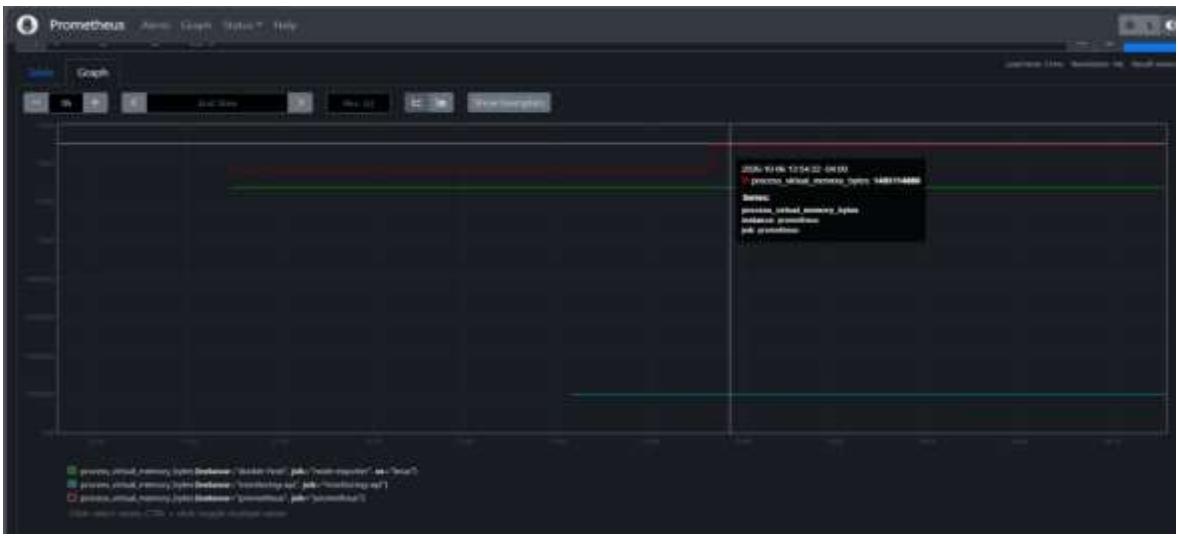
Tabla 4

Representación simplificada del diagrama de Gantt

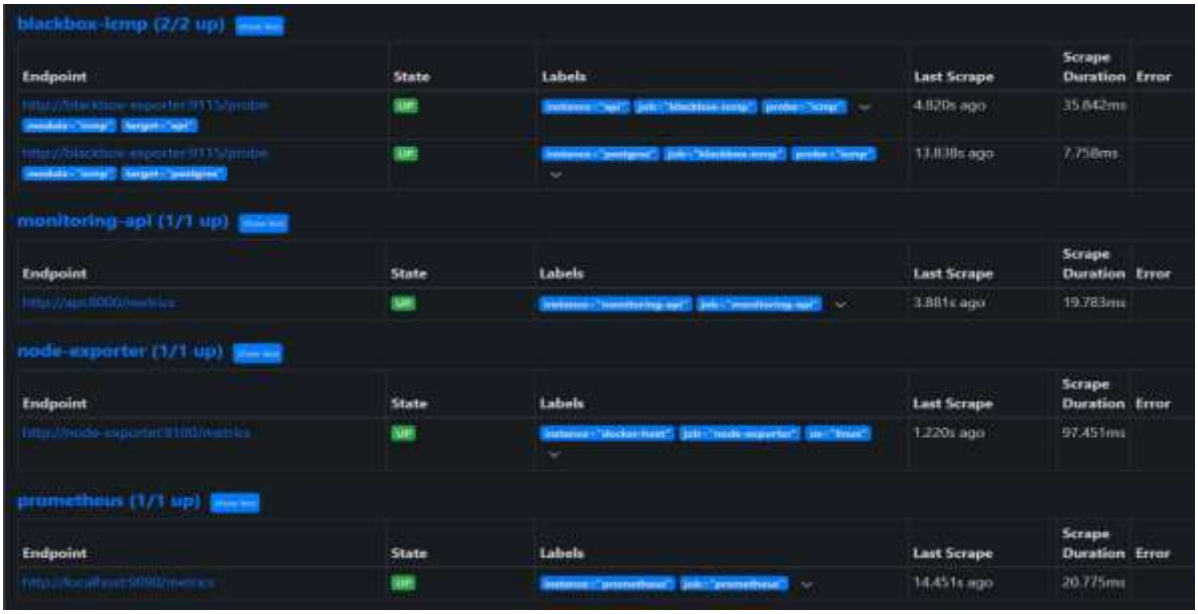
Actividad	Ago.	Sep.	Oct.	Nov.	Dic.
Investigación y definición					
Arquitectura					
Fase 1					
Fase 2					
Fase 3					
Fase 4					
Fase 5					
Fase 6					
Fase 7					
Fase 8					
Fase 9					
Fase 10					

Nota. Las celdas sombreadas indican los meses de ejecución de cada actividad. Elaboración propia.

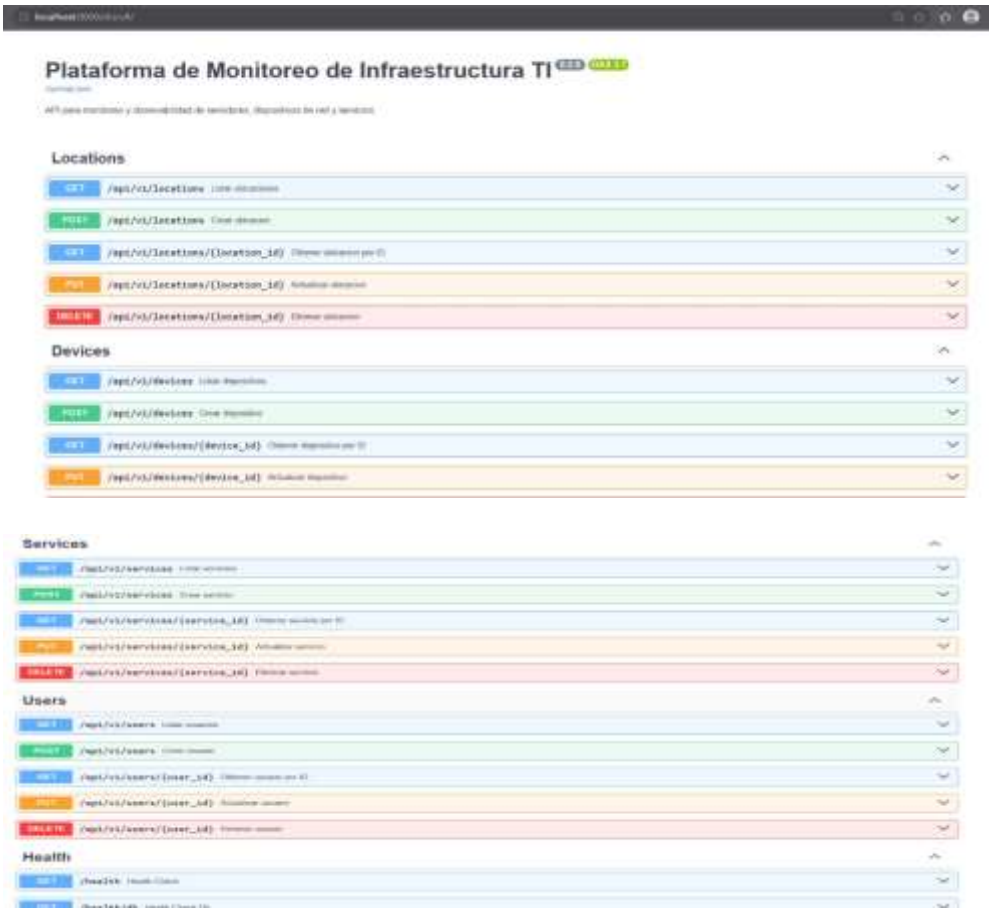
Anexo B. Prometheus consumo de RAM de los servidores



Anexo C. Prometheus com servicios activos



Anexo D. la interfaz de FastAPI



Anexo E. Preguntas y prompts utilizados con herramientas de inteligencia artificial

Pregunta/Prompt:
¿Qué herramientas de código abierto existen para implementar una plataforma de monitoreo de infraestructura TI y cuáles serían las más adecuadas para monitorear servidores Linux, servidores Windows, dispositivos de red y servicios?

Uso de la respuesta:
La respuesta fue utilizada como apoyo para comparar diferentes alternativas de monitoreo y determinar las tecnologías más adecuadas para el proyecto. A partir de esta investigación se seleccionaron principalmente Prometheus y Grafana, complementados con diferentes exporters según el tipo de recurso que se desea monitorear.

D.2 Diseño de la arquitectura

Pregunta/Prompt:
¿Cómo se podría diseñar una arquitectura de monitoreo utilizando Prometheus, Grafana, FastAPI, PostgreSQL y exporters para monitorear servidores, dispositivos de red y servicios?

Uso de la respuesta:
La información obtenida sirvió como referencia para organizar los componentes de la plataforma y definir la función de cada tecnología dentro de la arquitectura general. Posteriormente, la propuesta fue adaptada a las necesidades y alcance del proyecto.

D.3 Configuración de Prometheus

Pregunta/Prompt:
¿Cómo configurar Prometheus para realizar la recolección de métricas cada 15 segundos y utilizar exporters para obtener información de los sistemas monitoreados?

Uso de la respuesta:
La respuesta fue utilizada como guía para configurar el intervalo de recolección de métricas y organizar los diferentes objetivos de monitoreo. La configuración fue posteriormente comprobada dentro del entorno de desarrollo.

D.4 Desarrollo del backend

Pregunta/Prompt:
¿Cómo estructurar una API REST utilizando FastAPI para administrar dispositivos, servicios, reglas de monitoreo e información almacenada en PostgreSQL?

Uso de la respuesta:
La respuesta permitió obtener referencias para estructurar el backend, los endpoints y la comunicación con la base de datos. El código y las recomendaciones obtenidas fueron revisados y adaptados por el equipo antes de ser utilizados.

D.5 Base de datos

Pregunta/Prompt:
¿Qué información debería almacenarse en PostgreSQL para una plataforma de monitoreo de infraestructura TI?

Uso de la respuesta:
La consulta permitió diferenciar la información administrativa y estructurada que debía almacenarse en PostgreSQL de las métricas de series temporales que serían gestionadas por Prometheus.

D.6 Reglas de alerta

Pregunta/Prompt:
¿Qué reglas de alerta serían recomendables para una plataforma de monitoreo de infraestructura TI considerando CPU, memoria, almacenamiento, disponibilidad y otros recursos?

Uso de la respuesta:
Las recomendaciones fueron utilizadas como referencia para definir las condiciones de alerta del proyecto. Las reglas fueron posteriormente adaptadas a las necesidades de la plataforma y probadas durante la implementación.

D.7 Resolución de problemas técnicos

Pregunta/Prompt:
Analiza el siguiente error presentado durante la ejecución de un servicio mediante Docker Compose e indica posibles causas y soluciones: [mensaje de error].

Uso de la respuesta:
Las herramientas de IA fueron utilizadas como apoyo para interpretar errores y buscar posibles soluciones durante la implementación. Las recomendaciones fueron comprobadas por el equipo antes de realizar cambios en la configuración del proyecto.

Anexo F. Interfaz de Docker



Anexo G. Consultas de la base de datos PortgreSQL

```
C:\Users\Hp>docker exec -it monitoring-postgres psql -U monitoring_dev -d monitoring_db
psql (16.15)
Type "help" for help.

monitoring_db=# select*from locations
monitoring_db-# select * from locations;
ERROR:  syntax error at or near "select"
LINE 2: select * from locations;
        ^
monitoring_db=# SELECT * FROM locations;
                  id                  |          name          |       description       |     address      |
created_at           |          updated_at
--------------------------------------+------------------------+-------------------------+------------------+----------
--------------------+-------------------------------
 a9e32be1-754a-4376-9269-7f12e56d1bf8 | Data Center Principal | Sede central del campus | Edificio A, Piso 2 | 2026-10-0
6 17:15:30.199482+00 | 2026-10-06 17:15:30.19951+00
(1 row)

monitoring_db=#
```

```
monitoring_db=# \dt
                List of relations
 Schema |        Name         | Type  |      Owner
--------+---------------------+-------+------------------
 public | alembic_version     | table | monitoring_dev
 public | alert_assignments   | table | monitoring_dev
 public | alert_rules         | table | monitoring_dev
 public | audit_logs          | table | monitoring_dev
 public | devices             | table | monitoring_dev
 public | incidents           | table | monitoring_dev
 public | locations           | table | monitoring_dev
 public | notifications       | table | monitoring_dev
 public | services            | table | monitoring_dev
 public | users               | table | monitoring_dev
(10 rows)
```

Referencias

Docker. (s. f.). *Docker documentation*. https://docs.docker.com/

FastAPI. (s. f.). *FastAPI documentation*. https://fastapi.tiangolo.com/

Grafana Labs. (s. f.). *Grafana documentation*. https://grafana.com/docs/

Kim, G., Humble, J., Debois, P., y Willis, J. (2016). *The DevOps handbook: How to create world-class agility, reliability, and security in technology organizations*. IT Revolution Press.

PostgreSQL Global Development Group. (s. f.). *PostgreSQL documentation*. https://www.postgresql.org/docs/

Prometheus Authors. (s. f.). *Prometheus documentation*. https://prometheus.io/docs/

Python Software Foundation. (s. f.). *Python documentation*. https://docs.python.org/

Schwaber, K., y Sutherland, J. (2020). *The Scrum guide: The definitive guide to Scrum: The rules of the game*. https://scrumguides.org/scrum-guide.html

Zabbix. (s. f.). *Zabbix documentation*. https://www.zabbix.com/documentation/